Problem 1

The languages that are in P are:$DNFSAT$, $DNFUNSAT$, $CNFTAUT$, and $DNFTAUT$. To show this I will construct an algorithm of a decider for a multi-taped turing machine for each of these languages, all of which decide the language in a polynomial time.

$TM_{CNFTAUT}$: $O(n^2)$

1. Check if the input is in CNF, if it isn't then we reject it.
2. For each clause j do the following:
   2.1. For each literal l_i within the clause j do the following:
      2.1.1. Search for $\overline{l_i}$, if found go to step 2.
   2.2. Reject the input.
3. Accept the input.

The reason $TM_{CNFTAUT}$ is in $O(n^2)$ is because the total number of literal-literal pairs compared across the entire input is bounded by n^2, since there are at most n literals. Verifying whether the input is in CNF is in $O(n)$ since we are using a multi-taped turing machine to do this. Assuming we had a single-taped one instead it'd be at most in $O(n^2)$ due to the theorem 7.8 in Sipser's book. Similarly, $TM_{CNFTAUT}$ would be in $O(n^4)$.

$TM_{DNFUNSAT}$ is identical to the $TM_{CNFTAUT}$ except for the first step where it checks if the input is in DNF instead of CNF, and TM_{DNFSAT} does the opposite of what $TM_{DNFUNSAT}$ does. $TM_{DNFTAUT}$ is slightly different from them though. But I feel like I still need to point out explicitly exactly how they work.

$TM_{DNFUNSAT}$: $O(n^2)$

1. Check if the input is in DNF, if it isn't then we reject it.
2. For each clause j do the following:
   2.1. For each literal l_i within the clause j do the following:
      2.1.1. Search for $\overline{l_i}$, if found go to step 2.
   2.2. Reject the input.
3. Accept the input.

$TM_{DNFUNSAT}$ is also in $O(n^2)$ for exactly the same reason as to why $TM_{CNFTAUT}$ is.

TM_{DNFSAT}: $O(n^2)$

1. Check if the input is in DNF, if it isn't then we reject it.
2. Run $TM_{DNFUNSAT}$ with the same input.
   2.1. If $TM_{DNFUNSAT}$ accepts the input: Reject it.
3. Accept the input.

As you can see all this algorithm does is that it checks whether input is in DNF which we know is done in $O(n)$, and it utilizes $TM_{DFNUNSAT}$ which is in $O(n^2)$, so it is also in $O(n^2)$.

$TM_{DNFTAUT}$: $O(n^2)$

1. Check if the input is in DNF, if it isn't then we reject it.
2. For each clause j do the following:
   2.1. For each literal l_i within the clause j do the following:
      2.1.1. Search for $\overline{l_i}$, if found go to step 2.
   2.2. Accept the input.
3. Reject the input.

$TM_{DNFTAUT}$ is also in $O(n^2)$ for exactly the same reason as to why $TM_{CNFTAUT}$ is.

Problem 2

As for the languages that are NP-complete we only have one, and that is $CNFSAT$. To prove that it is NP-complete I will show an algorithm for a multi-taped turing machine that is able to verify a solution in polynomial time.

TM_{CNFSAT}: $O(n)$

1. Check if the input of the formula is in CNF, if it isn't then we reject it.
2. Check if the input of the certificate is valid, if it isn't then we reject it.
3. For each clause j do the following:
   3.1. For each literal l_i within the clause j do the following:
      3.1.1. Evaluate l_i using its corresponding certificate.
      3.1.2. If in step 3.1.1 l_i is evaluated as T then go to step 2.
   3.2. Reject the input.
4. Accept the input.

With this verifier we simply verify the inputs which is in $O(n)$, then we evaluate each literal or each clause which is also in $O(n)$. So the verifier is in $O(n)$ if a multi-taped TM, and in $O(n^2)$ if a single-taped one, both of which are polynomial. Thus, $CNFSAT$ is in NP.

However, to show that it is NP-complete I must also show that either of SAT or $3SAT$ (both of which are NP-complete) is p-reducable to $CNFSAT$. Between these two I choose to show that $SAT \leq_p CNFSAT$, and for that I define a function $f: SAT \rightarrow$ CNFSAT as follows:

First, it identifies all the biconditionals and implications and uses equivalences:
1. $x \rightarrow y \equiv \neg x \vee y$, and 2. $x \leftrightarrow y \equiv (\neg x \vee y) \wedge (x \vee \neg y)$ such that only conjunctions, disjunctions, and negations remain which is done in O(n). Next, it uses equivalences:
3. $\neg\neg x \equiv x$, 4. $\neg(x \wedge y) \equiv \neg x \vee \neg y$, and 3. $\neg(x \vee y) \equiv \neg x \wedge \neg y$ to transform it further such that all the negations are now only in front of literals and all the unnecessary negations are

gone, which is also in $O(n)$. Next it uses the equivalences: 5. $(a \vee b) \vee c \equiv a \vee b \vee c$ and 6. $(a \wedge b) \wedge c \equiv a \wedge b \wedge c$ to flatten any nested conjunctions and disjunctions, which is too in $O(n)$. Lastly, any conjunctions within the disjunctions are replaced with a fresh literal to prevent exponential growth which preserves equisatisfiability which is also in $O(n)$, so 7. $(a \wedge b) \vee c \; is \; equisatisfiable \; to \; x \vee c$ where x is a fresh variable. Using these the function achieves CNF in $O(n) + O(n) + O(n) + O(n) = O(n)$.

Now if the original SAT formula φ is satisfiable, then the formula transformed by the function f is also satisfiable due to the fact that only equivalences and equisatisfiable rules were used, i.e. $\varphi \; is \; satisfiable \Leftrightarrow f(\varphi) \; is \; satisfiable$. The fact that $SAT \leq_p CNFSAT$ via f which is in $O(n)$ and that SAT is NP-complete proves that $CNFSAT$ is NP-complete.

Problem 3

a) If $A \leq_p B$ then we have some function $f: A \to B$ in polynomial time such that $\varphi \in A \Leftrightarrow f(\varphi) \in B$. We can use that same function f as $\varphi \notin A \Leftrightarrow f(\varphi) \notin B$ which is equivalent to $\varphi \in \bar{A} \Leftrightarrow f(\varphi) \in \bar{B}$ due to the definition of $coNP$ being $coNP = \{\bar{A} \mid A \in NP\}$ and complement set being $\bar{A} = \{x \mid x \notin A\}$.

b) According to the definitions, a language A is NP-complete if A is in NP and A is p-reducable to every language B in NP, similarly a language $\bar{A}$ is $coNP$-complete if $\bar{A}$ is in $coNP$ and $\bar{A}$ is p-reducable to every language $\bar{B}$ in $coNP$. Since $A \; is \; NP \Leftrightarrow \bar{A} \; is \; coNP$ and $A \leq_p B \Leftrightarrow \bar{A} \leq_p \bar{B}$ from problem 3a, we can conclude that $A \; is \; NP$-complete $\Leftrightarrow \bar{A} \; is \; coNP$-complete. Thus, all the criteria for $\bar{A}$ being $coNP$-complete (and vice versa) are satisfied.

c) There is only one that is $coNP$-complete and that is $CNFUNSAT$ which is a complement of $CNFSAT$ because the negation of satisfiability is unsatisfiability. From problem 2 we know that $CNFSAT$ is NP-complete, and from problem 3b we know that A is NP-complete $\Leftrightarrow \bar{A} \; is \; NP$-complete. Thus, we conclude that $CNFUNSAT$ is $coNP$-complete.

Problem 4

a) Let's define a function $f: B \to A_{TM}$ where $f(\langle M \rangle) = \langle M, abb \rangle$. As you can see $\langle M \rangle \in B \Leftrightarrow \langle M, abb \rangle \in A_{TM}$ via f because since M accepts abb as an input, which is in B's definition, and A contains all the M's and the inputs they accept, $\langle M, abb \rangle \in A_{TM}$ must be the case.

b) Here we define a function $g: A_{TM} \to B$ where $g(\langle M, w \rangle) = M'$ where M' simply ignores its input and instead simulates M on the input w which is hardcoded into it. If M accepts w then M' accepts everything including abb, and if M does not accept w then M' rejects everything, including abb. This way both are either in their corresponding languages or not in them. Thus, $\langle M, abb \rangle \in A_{TM} \Leftrightarrow \langle M \rangle \in B$ via g.

c) We know that A_{TM} is not decidable but is recognizable which we had seen the proof of from before, and from problem 4b we got that $A_{TM} \leq_m B$ via g. That, and using the theorem 6.21 from Sipser's book, which states that "If $A \leq_m B$ is decidable and B is decidable then A is also decidable", we can conclude that B is not decidable because that would contradict the fact that A_{TM} is not decidable. Thus, B is recognizable just like A_{TM} is which we get from problem 4a's $B \leq_m A_{TM}$ and the theorem 6.22 from Sipser's book, which states that "If $A \leq_m B$ is recognizable and B is recognizable then A is also recognizable".